archive.today
webpage capture
Saved from https://levelup.gitconnected.com/using-linear-programming search 6 Feb 2025 08:50:05 UTC
no other snapshots from this url
Redirected from http://levelup.gitconnected.com/using-linear-programming-t
no other snapshots from this url
All snapshots from host levelup.gitconnected.com
Webpage Screenshot share download .zip report bug or abuse Buy me a coffee

Medium Search Write Sign up Sign in

Member-only story

Using Linear Programming to Solve the LinkedIn Queens Game

Example in Python PuLP

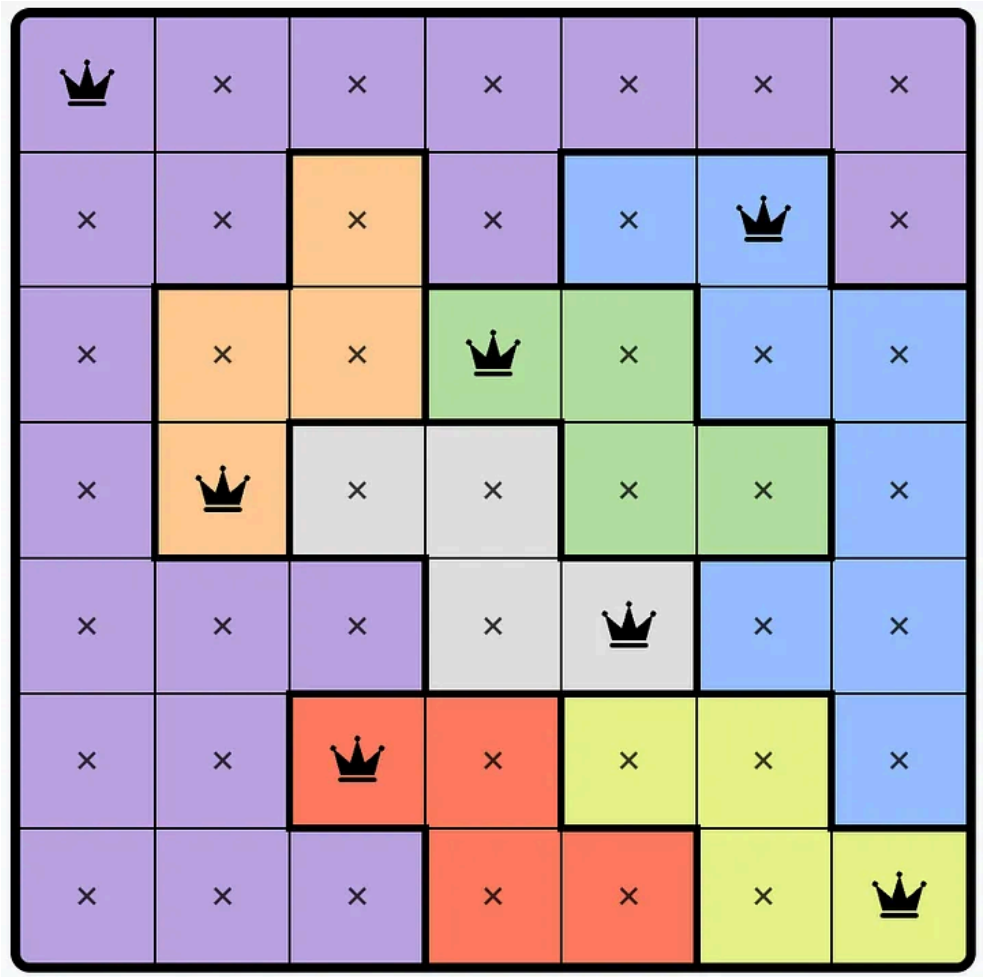
Brandon Walker · Follow
Published in Level Up Coding · 8 min read · Sep 12, 2024

64

The Queens Game

Queens is a small game that LinkedIn offers to its users, to get them to reopen the app/website. I feel like one of the Project Managers at LinkedIn in NYC is friends with one of the Project Managers in the games devision at the New York Times and they got jealous, thus giving me a problem to solve with programming. It is super fun to play as well.

Queens is played by trying to place Queens on a board, as pictured below. The constraints are that you have to put one Queen in every colored region, with no Queen being in the same row or column as any other, and no Queen touching another Queen, even diagonally.



The LinkedIn Queens Game from September 9th, 2024

Linear Programming

Linear Programming is a branch of mathematics that does constraint based optimization of problems that have linear constraints and objectives. Queens, like Sudoku has pretty clearly defined constraints and objectives, making both problems easy to apply Linear Programming to. Linear Programming isn't the only method of solving this, but I think it's actually one of the simplest.

Decision Variables

From the code's perspective, we treat each Queen as a 1, and each "x" or empty square as a 0. Each square has a decision variable, which the linear programming software seeks to optimize. The point of any linear programming optimization problem is to have the program tell you the optimal number in each decision variable.

Objective

The objective is to maximize the total number of 1s on the board. So the sum of the values of each of the squares (ie the sum all the decision variables) is

known as the objective function.

Constraints

Without any constraints, maximizing the objective function would place a 1 in every square, so we have to add code that represent the rules of the game. First the sum of a row can't be more than 1. Second, the sum of a column can't be more than 1. Third, the sum of any 2x2 box can't be more than 1 (this prevents diagonally touching queens). Finally, the sum of a color group can't be more than 1.

This is the basics of linear programming, how the solver works under the hood is a more complicated, and I admittedly don't know too much about it. I generally think you don't need to know how the solver works, as long as you can write your objective function, decision variables, and constraints well, this is taken care of by the software you're using.

The Code

I did this with an open source python linear programming package called PuLP. It's certainly not the only software that is capable of doing this, there are other tools and languages that have these capabilities that are both open and closed source.

All of the python code is encapsulated in just one file that is under 80 lines including comments and empty lines. If you get lost and want to see the finished code and run that yourself it's here on my GitHub.

Set Up

Other than PuLP I only used one python packages to solve this, Beautiful Soup 4, in order to read the html elements on the page of the Queens game. This is necessary to get access to the size of the board and the colors of each cell. Both packages can be found on PyPi along with installation instructions.

Imports and Helper Functions

One lazy thing I did is have a global variable indicating the number of rows and columns on the board called `number_of_rows` which could be calculated automatically by looking at the HTML elements. In the problem pictured above it was 7, but all games will always have a minimum of 5 rows/columns.

```
number_of_rows = 7
```

The Queens game under the hood has IDs for each cell starting at 0. This was probably convenient for checking on the backend if the solutions was solved. I prefer to think of every square as being identified by two IDs together indicating its row and column. But since LinkedIn wrote it using just one ID for each cell, I first write a function that take row and column integer arguments and calculates the ID from it.

```
def get_cell_id(row, col):
    return row*number_of_rows + col
```

Another helper function is returning the the IDs of the cells to the immediate right, below, and to the lower right of a starting cell. This is helpful later on to ensure that in any 2x2 square, there is only one Queen. If there were more than one, it would violate the no touching of Queens diagonally rule. I don't think you have to understand this bit of code super well, if you'd like to skip over it but just know what it does, I think that is fine.

```
def get_surrounding_cell_ids(row, col):
    starting_cell_id = get_cell_id(row, col)

    # right
    if col+1<number_of_rows: # checks for overflow (ie edge of the board)
        right = get_cell_id(row, col+1)
    else:
        right = None

    # bottom
    if row+1<number_of_rows: # checks for overflow (ie edge of the board)
        bottom = get_cell_id(row+1, col)
        if col+1<number_of_rows: # checks for overflow (ie edge of the board)
            bottom_right = get_cell_id(row+1, col+1)
        else:
            bottom_right = None
    else:
        bottom = None
        bottom_right = None

    # list of the ids
    ids = [starting_cell_id,   right,
           bottom,             bottom_right]

    # remove the Nones so constraint only has actual IDs
    ids = [id for id in ids if id is not None]
    return ids
```

Defining the Problem in PuLP

All the code we need from PuLP can be imported as below.

```
from pulp import LpProblem, LpVariable, LpMaximize, value
```

Now we create a PuLP problem, and describe the way we are trying to optimize the objective function. I give my problem the arbitrary name Queens below, and set the objective of maximization. You can also set this to minimize by importing `LpMinimize` on other problems.

```
problem = LpProblem("Queens", LpMaximize)
```

Define the Variables

The decision variables are each stored in a dictionary I call `variables` and are created with the `LpVariable` class. I iterate across every row and column and store them in this dictionary by their ID. Within the instantiation of the `LpVariable` you need to say what type of variable it is, which can have the value “Continuous”, “Integer”, or “Binary” depending on the problem type.

```
# define decision variables
variables = {}
for row in range(number_of_rows):
    for col in range(number_of_rows):
        cell_id = get_cell_id(row, col)
        variables[cell_id] = LpVariable(f"var_{cell_id}", cat="Binary")
```

Define the Constraints

Here is where we code our constraints. Remember that they are:

1. Only one queen per row
2. Only one queen per column
3. Only one queen in any 2x2 box (ie no diagonally touching)
4. Only one queen per color

The first two constraints are written pretty easily by iterating across every row and column. Since the board is a square the variable `number_of_rows` gets used as the number of columns as well. We say that the sum of the decision variables must exactly equal 1, and then we add this constraint to the problem.

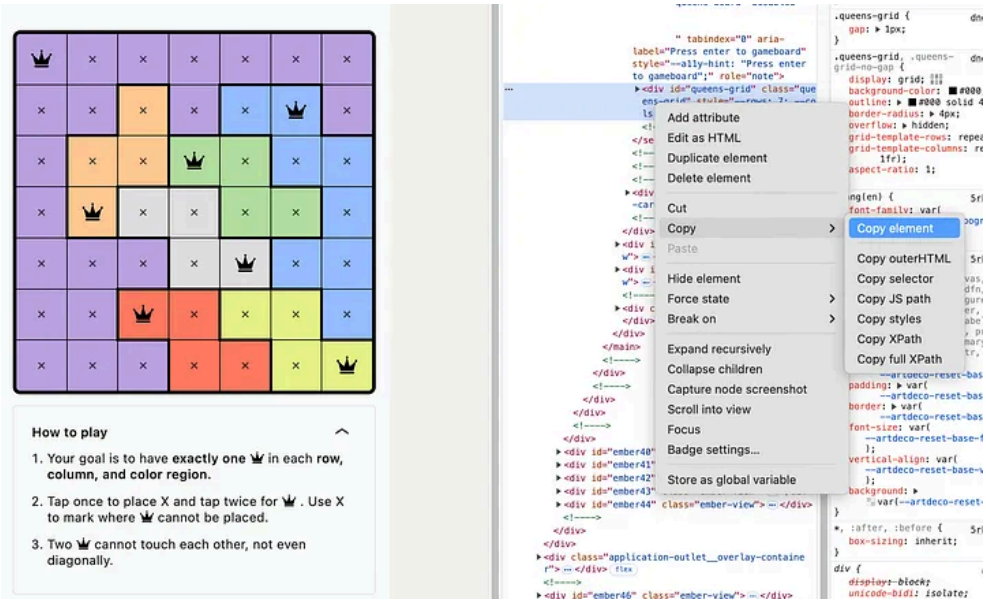
```
# constraints for only 1 queen in a row
for row in range(number_of_rows):
    problem += sum([variables[get_cell_id(row, col)] for col in range(number_of_

# constraints for only 1 queen in a column
for col in range(number_of_rows):
    problem += sum([variables[col+row*number_of_rows] for row in range(number_of
```

For the constraint of only one queen in any 2x2 box, we can iterate across every ID, get the surrounding IDs and say that the sum of these four decision variables must be less than or equal to 1. Not all 2x2 boxes will have even one Queen, but they can't have two, three, or four. There is definitely other ways of writing this diagonal constraint, but it was the first thing I thought of, and the code functions relatively simply.

```
# no surrounding 1s
for row in range(number_of_rows):
    for col in range(number_of_rows):
        problem += sum([variables[var] for var in get_surrounding_cell_ids(row,
```

The color constraint is a bit more tricky, as it requires you to read from LinkedIn's website the HTML elements that define the game. This is hard to do in an automated from code because you have to send a request for the page, while authenticated. Since the point of the exercise is doing the LP, and I didn't want to mess around with anything auth related since I am putting this on GitHub publicly, I just copy the HTML element that has the class "queens-grid" after inspecting the page then store the result in a file next to my python script and called `game.html`.



The colors are stored in div elements that have the class queens-cell, with a second class of that element indicating the color. Opening the `game.html` file and finding these elements can be done with the code below.

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(open("game.html"))
color_groups = [i["class"][1] for i in soup.find_all("div", {"class": "queens-ce
```

The variable `color_groups` now contains a list of colors. Position <x> in the list has the value of the color of cell id <x>.

By iterating across the distinct colors, and then iterating across all the cell IDs we add the decision variables to a constraint where the total count of Queens must be equal to 1.

```
for color in set(color_groups): # iterate across all colors, this
    color_group_constraint = []
    for cell_id,cell_color in enumerate(color_groups):
        if cell_color==color: # if cell_color
            color_group_constraint.append(variables[cell_id])
    problem += sum(color_group_constraint)==1
```

Objective Function

This is the bit of code we add to tell the Linear Programming solution to maximize the number of Queens on the board. It looks a lot like a constraint, except it has no comparison operator.

```
problem += sum([variables[var] for var in variables])
```

Solve the Optimization

Once all objective functions and constraints have been added, running the solve function on the problem calculates the results.

```
solution = problem.solve()
```

The solution will tell you how long the solver took, the value of your objective function once it has been optimized, and if it could find a solution at all. In our problem the objective value should be 7, the number of Queens on the board which we were trying to maximize.

```
Result - Optimal solution found

Objective value:                7.00000000
Enumerated nodes:               0
Total iterations:               0
Time (CPU seconds):             0.00
Time (Wallclock seconds):       0.02
```

Print Out Solution

This next block of code prints out the solution. The LpVariables stored in the variables dict have now been optimized by the solver and will contain 0s and 1s now. The `value` function from pulp is used to access the value.

```
for row in range(number_of_rows):
    for col in range(number_of_rows):
        cell_id = get_cell_id(row, col)
        print(int(value(variables[cell_id])), end =" ")
    print()
```

The resulting values printed out looks like the bit below. You'll notice where there are 1s there were also Queens in the image above.

```
1 0 0 0 0 0 0
0 0 0 0 0 1 0
```

```
0 0 0 1 0 0 0
0 1 0 0 0 0 0
0 0 0 0 1 0 0
0 0 1 0 0 0 0
0 0 0 0 0 0 1
```

Conclusion

I think this is a overlooked branch of data science, many problems out there are not ML based, but are pretty simple optimization with already understood constraints. It can be applied to scheduling, planning, economics, and a wide variety of business problems.

There is plenty of other options out there for linear programming software, I am aware of Microsoft, IBM, and Google all having offerings around this problem space. I first worked of operations research type problems in college, but I started doing it in a practical sense while I worked at IBM.

Data Science | Operations Management | Games | Linear Programming | LinkedIn

64

Published in Level Up Coding

Follow

199.6K Followers · Last published 1 day ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Written by Brandon Walker

Follow

1.4K Followers · 79 Following

Machine Learning DevOps Engineer at John Deere

No responses yet

What are your thoughts?

Respond

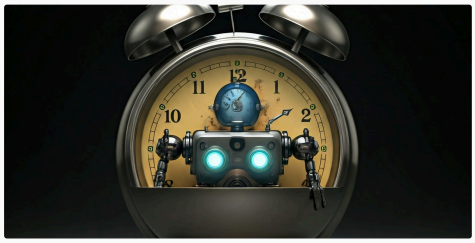
More from Brandon Walker and Level Up Coding

In Level Up Coding by Brandon Walker

6 Tips for Making AWS Lambda Cheaper

A General Guide, with Some Examples in Python

Sep 8, 2024 205

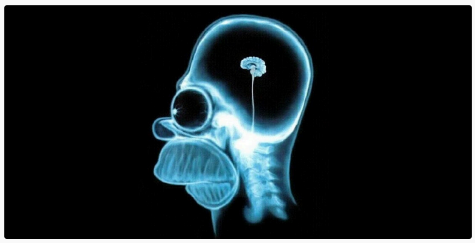


In Level Up Coding by Yusuf Sarıgöz

Open-source implementation of OpenAI's Tasks feature

Let LLMs schedule tasks and reminders for you and itself

1d ago 7

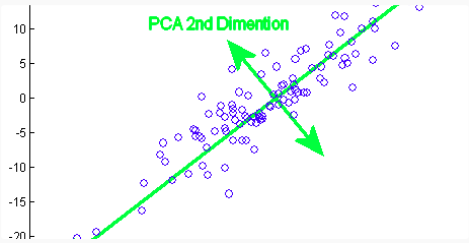


In Level Up Coding by Fareed Khan

Coding the Brain of an LLM to See How It Thinks

Debugging One layer Transformer

1d ago 212 2



In Towards Data Science by Brandon Walker

PCA Is Not Feature Selection

What it actually does and when you can and can't use it.

Dec 31, 2019 567 5

See all from Brandon Walker

See all from Level Up Coding

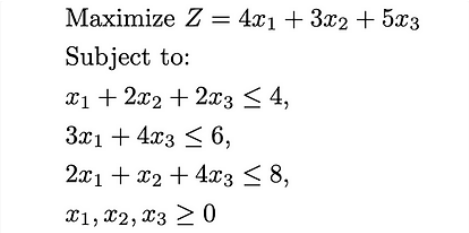
Recommended from Medium

In Write A Catalyst by Onyedikachukwu Czar

DeepSeek Just Confirmed My Suspicions About OpenAI

The ChatGPT maker has been playing a losing game

Jan 28 2.8K 124



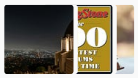
Min Kyung Lee

Linear Programming: Sensitivity Analysis Part 1

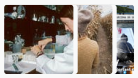
Linear programming (LP) is a powerful tool for optimizing decision-making, but the...

Nov 30, 2024 2

Lists

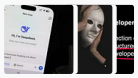


My Kind Of Medium (All-Time Faves)
111 stories · 650 saves



Medium's Huge List of Publications Accepting...
414 stories · 4484 saves

Staff picks
807 stories · 1610 saves



Natural Language Processing
1918 stories · 1575 saves

In Generative AI by Jim Clyde Monge

DeepSeek Releases Its Own AI Image Generator, Janus-Pro

DeepSeek says Janus-Pro 7B outperforms OpenAI's Dall-E 3 and Stable Diffusion in...

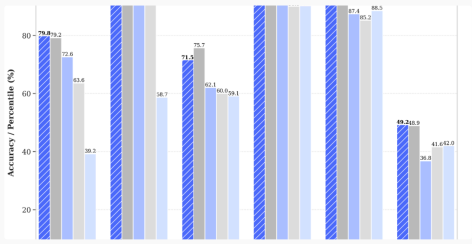
Jan 28 1.5K 55

Alberto Romero

DeepSeek Is Chinese But Its AI Models Are From Another Planet

OpenAI and the US are in deep trouble

Jan 22 6.1K 193



In Level Up Coding by Joshua Saunders

DeepSeek Has Arrived In Cursor — But Is It Better Than Claude?

Chat and Composer mode tests for all three models to see if a new king is crowned

Jan 27 356 9

Jessica Stillman

Jeff Bezos Says the 1-Hour Rule Makes Him Smarter. New...

Jeff Bezos's morning routine has long included the one-hour rule. New...

Oct 30, 2024 22K 615

See more recommendations

Help Status About Careers Press Blog Privacy Terms Text to speech Teams